

DESIGN OF THE IBM GRANITE 5.0 TURBOCTC ASR MODEL

Brian Kingsbury, George Saon, Masayuki Suzuki, Hong-Kwang J. Kuo
Takashi Fukuda, Samuel Thomas, Vishal Sunder, Avihu Dekel

IBM Research

ABSTRACT

We describe the architecture, training methodology and inference speedups of Granite 5.0 Turbo CTC, a 470 million parameter encoder-only model with an excellent speed-accuracy tradeoff. The architecture uses pyramidal temporal subsampling within Conformer blocks using strided depthwise convolutions, block-diagonal (chunk-wise) self-attention, and conditioning on intermediate predictions from the middle layer. Training highlights are the use of only publicly available data, the novel use of a Muon optimizer, and balanced data sampling. Inference speedups include replacing 1×1 convolutions with linear layers and optimizing the attention computation in the Conformer blocks. Collectively, these result in a model that is on the speed-accuracy Pareto frontier of the Open ASR leaderboard for English short-form ASR while being twice as fast as the fastest competitor. The model can be used under a permissive license and downloaded from <https://huggingface.co/ibm-granite/granite-speech-5.0-470m-turboctc>.

Index Terms— speech recognition

1. INTRODUCTION

Recent improvements in automatic speech recognition (ASR) have been driven by the integration of text-based large language models (LLMs) into ASR systems. One popular approach to integration uses a separately trained acoustic encoder, connects the encoder to the LLM via a trainable projector, and equips the LLM with low rank adapters [1, 2]. Such speech language models enjoy capabilities like speech translation, contextual biasing, and seamless integration of dialog context as well as strong transcription accuracy [3–7].

However, LLM integration is costly because the LLM brings in a large number of parameters, driving up memory footprint, and the autoregressive generation process used in most LLMs limits transcription speed. Moreover, a well-trained Conformer model is a strong ASR model in its own right [8]. Therefore, in this paper we focus on encoder-only speech recognition using a Conformer [9] model trained using the connectionist temporal classification (CTC) [10] loss. We make the following contributions in this paper.

- We use strided depthwise convolutions in the Conformer blocks to implement pyramidal temporal subsampling.
- We apply Muon with Polar Express orthogonalization to train the base model.
- We accelerate inference by replacing 1×1 convolutions with linear layers and optimizing the attention computations in the Conformer blocks.
- We improve the accuracy of the model by distilling knowledge from an LLM and by performing robust fine tuning.

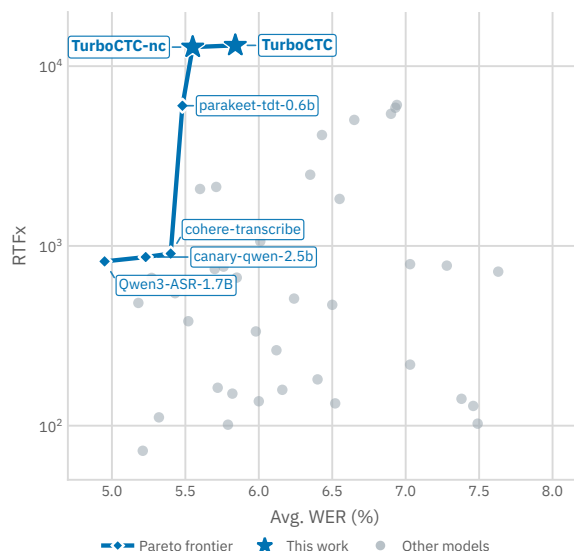


Fig. 1. OpenASR leaderboard speed-accuracy Pareto curve as of 4 September 2026. Higher RTFx and lower WER are better; granite-speech-5.0-470m-turboctc and granite-speech-5.0-470m-turboctc-nc are marked with stars. The axes are cropped for legibility, placing 6 of the 51 models outside the view; the entire Pareto frontier is shown.

These contributions are verified in a strong model that was the fastest on the OpenASR Leaderboard [11] as of 4 September 2026 and resided on the speed-accuracy Pareto frontier as shown in Figure 1. A companion model that had additional training data, granite-speech-5.0-470m-turboctc-nc, was only 0.7% behind the top model, a speech language model, in accuracy. Owing to their encoder-only design, both models are extremely small, requiring only 470 million parameters.

2. ARCHITECTURE

In this section we detail the architectural design elements for granite-speech-5.0-470m-turboctc pertaining to temporal downsampling, block attention and conditioning on intermediate predictions. The model consists of 16 Conformer blocks with a hidden dimension of 1024 and an output layer of size 16,384 for a total of 470 million parameters. Other architectural details are shown in Table 1.

Temporal downsampling is achieved in two separate steps. In step 1, we reduce the log-Mel frame rate from 100 Hz to 50 Hz by stacking every two consecutive frames and skipping every other frame

Configuration parameter	Value
Input dimension	320 (80 logmel + 80 deltas) x 2
Number of layers	16
Hidden dimension	1024
Number of attention heads	8
Attention head size	128
Attention block size	128
Convolution kernel size	7
Output dimension	16384

Table 1. Configuration parameters for the model architecture.

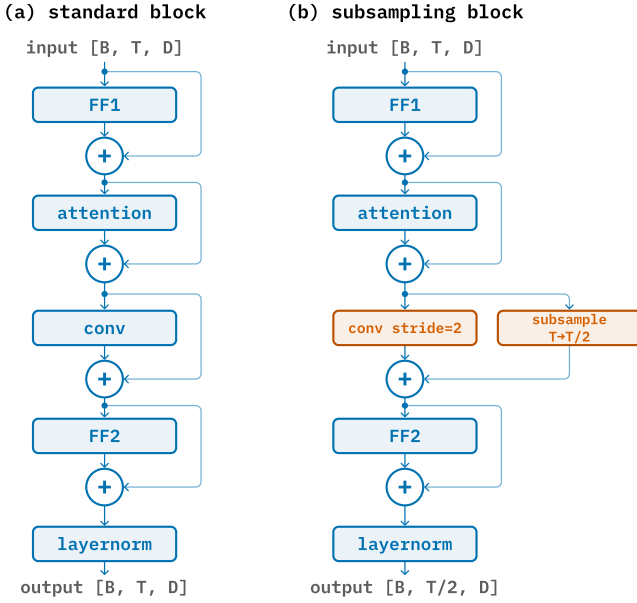


Fig. 2. Comparison of standard and subsampling Conformer blocks.

through a simple tensor reshape operation. In step 2, the first two Conformer blocks perform temporal downsampling, each achieving a 2x temporal reduction (total of 4x) resulting in a 12.5 Hz frame/token rate that gets propagated through the remaining Conformer blocks and output layer. Inspired by [12], we change the depthwise convolutions in the Conformer blocks to stride 2 convolutions. In order to match the sequence length of the residual coming out of the self-attention layer to the resulting half-length sequence from the convolutional layer, we also perform average pooling of every 2 consecutive residual attention frames. The difference between standard and subsampling Conformer blocks is illustrated in Figure 2. To compensate for the reduced token rate, we use an output layer with 16K BPE subword units derived from the acoustic transcripts. In Table 2, we compare our subsampling approach with purely convolutional subsampling as proposed in the FastConformer architecture [13] on the public English test sets of the Open ASR leaderboard for the same 8x temporal reduction (rows 1 and 2).

Secondly, we opt for block-diagonal (or chunkwise) self-attention in the Conformer layers. This has two advantages over full attention: (i) the complexity scales linearly instead of quadratically with the length of the audio sequence and (ii) it generalizes better to long-

form audio based on the findings of [14]. In practice, we use chunk sizes of 128 frames for all Conformer layers (including the first two subsampling layers) which corresponds to 2.5 seconds (layer 1), 5 seconds (layer 2) and 10 seconds (layers 2-16) of audio. In the same Table 2, we compare block attention with grouped attention proposed in [12] where for the latter we used 3 groups for the first 6 conformer layers (rows 1 and 3).

Lastly, we use conditioning on intermediate predictions from the middle Conformer layer as proposed in [15]. The frame-level softmax distributions from the 8th layer are linearly mapped back to the hidden dimension and added as a residual connection to the input of the 9th layer. The output layers are tied between the middle and last layers and the CTC loss is computed as a weighted sum between the intermediate and last layer CTC losses with weights 0.2 and 0.8, respectively. The importance of self-conditioning is shown in Table 2 (rows 1 and 4) where, if we train a model without it, the performance degrades across all test sets.

3. TRAINING

The model is trained in three phases: (1) base model training with the CTC loss, (2) knowledge distillation from a Granite LLM, and (3) robust fine tuning. Table 3 shows how each stage of training improves the accuracy of the model. The following subsections describe these stages in more detail.

3.1. Base model training

The model is trained on the approximately 60k hours of audio from publicly available English ASR corpora and synthetic data listed in Table 4. **Multispeaker** is synthesized by concatenating single-speaker segments sampled from MLS, YODAS, CommonVoice-17, VoxPopuli, and AMI. **Multispeaker-Earnings** is synthesized by concatenating single-speaker segments sampled from Earnings-22. **Numbers** comprises a collection of utterances containing numeric expressions, phone numbers, monetary values, e-mail addresses, URLs, and street addresses. The utterances were generated using gpt-oss-120b [23] or gpt-oss-20b and synthesized with a StyleTTS 2 [24] model.

Initial model development was done using AdamW [25], but as described below we switched to a Muon [26] variant for training the final model. Other training hyperparameters were set as follows.

- Training on a compute node with 8 NVIDIA H100 GPUs.
- Balanced data sampling [6] with balancing parameter $\alpha = 0.8$ and up to 448 seconds of audio per GPU.
- Weight initialization uses PyTorch 2.11 defaults except for the output biases and intermediate projection weights and biases, which are initialized to 0.0.
- PyTorch OneCycleLR learning rate schedule, linear warmup from 5×10^{-5} to 5×10^{-4} over 177k updates, linear annealing to 1.5×10^{-7} over 1.71M updates, and default momentum cycling on both Muon’s momentum and AdamW’s β .
- The gradient $L2$ norm is clipped to 10.0.
- Weight decay with $\lambda = 0.01$ is applied in AdamW and Muon, excluding biases and normalization parameters.
- Dropout [27] is applied inside the Conformer blocks with $p = 0.1$ to the outputs of the multihead attention, convolution, and feedforward blocks, with $p = 0.1$ to the hidden layer in the feedforward blocks, and with $p = 0.25$ before the output projection.

Architecture	AMI	Earnings22	Gigaspeech	LS-clean	LS-other	SPGI	VoxPopuli	AVG
Proposed	8.53	8.65	10.17	1.34	2.63	3.37	6.13	5.83
Convolutional front end [13]	8.71	8.68	10.22	1.37	2.72	3.58	6.15	5.92
Grouped attention [12]	8.96	8.79	10.21	1.37	2.69	3.38	6.15	5.94
No self-conditioning [15]	8.89	9.72	10.28	1.46	2.85	3.55	6.25	6.00

Table 2. WER performance on the public components of the OpenASR Leaderboard for different approaches to temporal subsampling, attention and self-conditioning.

Training stage	AMI	Earnings22	Gigaspeech	LS-clean	LS-other	SPGI	VoxPopuli	AVG
Base	8.53	8.65	10.17	1.34	2.63	3.37	6.13	5.83
LLM distillation	8.51	8.62	9.95	1.33	2.56	3.28	5.98	5.75
Robust fine tuning	8.32	8.54	9.93	1.32	2.54	3.35	6.03	5.72

Table 3. WER performance on the public components of the OpenASR Leaderboard for the three training stages.

Dataset	Source	# hours
MLS [16]	natural	44600
YODAS [17]	natural	8900
CommonVoice-17 [18]	natural	2500
Librispeech [19]	natural	960
VoxPopuli [20]	natural	500
AMI [21]	natural	150
Earnings-22 [22]	natural	100
Multispeaker	synthetic	2000
Multispeaker-Earnings	synthetic	500
Numbers	synthetic	240

Table 4. List of datasets used to train `granite-speech-5.0-470m-turboctc`.

- SpecAugment [28] is applied to the input features. The frequency masks are up to 15 filters, the number of frequency masks is sampled uniformly from $\{1, 2\}$, and with $p = 0.10$ no frequency masking is applied. The time masks are up to 24 frames long and the number of time masks is sampled uniformly from $\{0, \dots, \lfloor 0.03125 * T \rfloor\}$, where T is the length of the utterance in frames. Identical time and frequency masking are applied to the static and delta feature streams.
- Noise augmentation is applied to the input audio. We add noise samples with $p = 0.25$ where the signal-to-noise ratio is sampled from $\mathcal{U}[-5, 20]$ dB.

We investigated three different optimization algorithms for training the base model: (1) AdamW, (2) Muon, and (3) Muon with Polar Express (MuonP for short) [29]. Muon, which has recently been used to train frontier LLMs [30], optimizes **matrix-structured** parameters via steepest descent in the spectral norm [31], relying on iterative methods to approximate the necessary polar decomposition of the update matrix. Because Muon and MuonP apply only to two-dimensional parameters that act as linear operators, we rely on AdamW to train the biases, normalization parameters, relative positional embeddings, temporal convolutions, output projection, and the map from intermediate token posteriors back to the internal dimension of the Conformer stack. We use the update scaling and weight decay from [30].

Table 5 compares the performance of the three optimizers on the public parts of the 20 May 2026 version of the OpenASR leaderboard.

All hyperparameter settings are as listed above for all three training runs. Both Muon variants outperform AdamW in aggregate word error rate, and the Polar Express version of Muon outperforms AdamW on all test sets. The MuonP-trained Conformer thus became the base for `granite-speech-5.0-470m-turboctc`.

3.2. LLM Knowledge Distillation

We train an auxiliary 4-layer, 1024 hiddens transformer decoder with 75 million parameters used to predict the output distribution of the `granite-4.1-8b-base` text LLM. The decoder cross-attends to the encoder embeddings from the last layer and is conditioned on the previous token embeddings with teacher forcing during training. The distillation loss measures the KL divergence between the per-token teacher softmax distribution truncated to the top 10 tokens and the student distribution where the latter shares the frozen `granite-4.1-8b-base` output layer of size 100 K BPE units. Importantly, the AED decoder is only used for training and discarded at inference. Training is done with the MuonP optimizer on the acoustic training data from Table 4 over 5 epochs with linear annealing using separate maximum learning rates for the CTC pre-trained encoder (1e-6) and AED decoder (1e-3). The combined training loss is a weighted sum of the per-frame CTC loss and the distillation loss with weights of 0.3 and 0.7, respectively. The improvements in accuracy due to this step are shown in Table 3 (rows 1 and 2).

3.3. Robust Fine-Tuning

To improve robustness to acoustic distortions while preserving recognition accuracy on clean speech, we combine multi-condition fine-tuning on paired clean and acoustically perturbed speech with parameter interpolation between the original and adapted models. For each training utterance x with transcription y , we first apply speed perturbation, sampling the factor uniformly from $\{0.8, 0.9, 1.0, 1.1, 1.2\}$. The resulting waveform serves as the clean view x_c , and a copy is further perturbed to produce the distorted view x_d . Both views share the same transcription. The clean view receives no added noise, reverberation, bandwidth filtering, or codec processing.

To construct the distorted view, we independently apply additive background noise, reverberation, bandwidth filtering, and codec and quantization processing with probabilities 0.25, 0.50, 0.30, and 0.15, respectively. When noise is applied, the SNR is sampled uniformly from $[-5, 20]$ dB. Noise recordings are drawn from the

optimizer	AMI	Earnings22	Gigaspeech	LS-clean	LS-other	SPGI	VoxPopuli	AVG
AdamW	8.66	8.89	10.21	1.37	2.75	3.65	6.41	5.99
Muon	8.67	8.88	10.04	1.40	2.65	3.43	6.44	5.93
MuonP	8.53	8.65	10.17	1.34	2.63	3.37	6.13	5.83

Table 5. WER performance on the public components of the OpenASR Leaderboard for different optimizers.

noise subset of MUSAN [32], which contains recordings originating from Freesound and SoundBible. Each noise recording is randomly cropped, with repetition where necessary, to match the utterance duration.

Reverberation is generated by convolving the waveform with measured or simulated room impulse responses from the Room Impulse Response and Noise Database (OpenSLR SLR28) [33], including responses from the REVERB Challenge 2014 dataset and the Aachen Impulse Response Database. These responses span a range of acoustic conditions, including far-microphone configurations, large rooms, lecture rooms, meeting rooms, offices, and a highly reverberant hall. The convolved waveform is cropped to the original duration and rescaled to preserve its pre-convolution peak amplitude.

Bandwidth perturbations use high-pass, low-pass, and band-pass filters, including telephone-band filtering at 300–3400 Hz and more restrictive conditions. Channel perturbations include PCM quantization, μ -law, A-law, GSM, and ADPCM processing, together with low-pass filtering and quantization to approximate additional lossy channel effects.

The model is trained on the paired views using a weighted combination of CTC losses:

$$\mathcal{L}_{\text{robust}}(\theta; x, y) = (1 - w)\mathcal{L}_{\text{CTC}}(\theta; x_c, y) + w\mathcal{L}_{\text{CTC}}(\theta; x_d, y), \quad (1)$$

where $w \in [0, 1]$ controls the relative contribution of the distorted-view loss. We select $w = 0.5$ for the final configuration. The clean-view loss helps preserve recognition accuracy on clean speech, while the distorted-view loss encourages robustness to acoustic distortions. We also experimented with an additional consistency loss between the predictions for the clean and distorted views, but omitted it from the final objective because it did not improve recognition performance.

We fine-tune the full CTC model for one epoch on the same speech training data using the combined loss from the clean and distorted views. The initial learning rate is 10^{-5} and is linearly annealed to 10^{-7} after a short initial plateau. Balanced sampling uses a balancing parameter of 0.9; the optimizer configuration and other regularization settings are retained from base model training.

Finally, to balance robustness gains with the preservation of clean-speech recognition accuracy, we interpolate the model parameters before and after robust fine-tuning:

$$\theta_{\text{interp}} = (1 - \gamma)\theta_0 + \gamma\theta_r, \quad (2)$$

where θ_0 denotes the model before adaptation and θ_r the robust fine-tuned model, with $\gamma = 0.4$ in practice. Interpolation produces a single model with unchanged architecture and inference cost.

Robust fine-tuning contributed to our models being ranked in the top 10 by accuracy on the noisy and far-field ASR (FFASR) leaderboard [34] while having the two highest inference speeds.

4. SPEEDUPS

Our Conformer code is based on a well-known open-source implementation [35] with several modifications. We implemented chunked attention [14] to avoid quadratic memory and compute scaling,

efficient?	linear?	RTFx	speedup (%)
×	×	9229	0
✓	×	11369	23
×	✓	10688	16
✓	✓	13715	49

Table 6. Inverse real time factors for the baseline model and models using one or both of our speedups.

we replaced the `einsum`-based content attention with PyTorch’s `scaled_dot_product_attention`, and we implemented masking to prevent padded locations from affecting the model’s output.

On top of this baseline, we identified two code changes that accelerated training and inference. First, we observed that, due to the way the attention operation was organized, the `MATH` backend was always invoked. Reorganization of this computation permitted consistent invocation of the `EFFICIENT_ATTENTION` backend instead. Note that the use of Shaw’s relative positional encoding precludes the use of `FlashAttention`. Second, we replaced the 1×1 convolutions in the Conformer’s depthwise convolution block with mathematically equivalent `Linear` operations after we found that the corresponding kernels were more efficient.

Table 6 reports inverse real time factors (RTFx) for the public datasets on the 20 May 2026 version of the OpenASR leaderboard. The measurements were made with a batch size of 128 utterances using a single NVIDIA H100 GPU on a Dell XE9680 node running PyTorch 2.6.0 with CUDA 12.4. These tests were done using a model checkpoint from the end of the first phase of training, prior to LLM knowledge distillation and robust fine tuning, with greedy decoding. Each of the four model runs achieves an aggregate 5.83% WER. Taken together, the efficient attention kernel and use of `Linear` operations instead of `Conv1d` operations speed up inference by 49%, with the larger gain coming from the efficient attention. We observed less dramatic speedups in training because that process involves other operations that are not accelerated by these changes.

5. CONCLUSIONS

We have described `granite-speech-5.0-470m-turboctc` and shown how specific architectural choices, self-conditioning and temporal downsampling via strided depthwise convolutions in the first two conformer blocks; training methods, the use of Muon for base training, distillation from a Granite LLM, and robust fine tuning; and inference speedups, replacing 1×1 convolutions with linear layers and optimizing attention kernels, combine to produce an ASR model that is on the speed-accuracy Pareto frontier of the Open ASR leaderboard for English short-form ASR while being twice as fast as the fastest competitor.

6. USE OF AI

Claude Code was used to aid in writing portions of the training and inference code and in the preparation of Figures 1 and 2. All experiments in this paper were run and verified by hand. The authors are fully responsible for the content of this submission.

7. REFERENCES

- [1] C. Tang, W. Yu, G. Sun et al., “SALMONN: Towards generic hearing abilities for large language models,” in *Proc. ICLR*, 2024.
- [2] Z. Ma, G. Yang, Y. Yang et al., “Speech recognition meets large language model: Benchmarking, models, and exploration,” in *Proc. AAAI*, 2025.
- [3] A. Grattafiori, A. Dubey, A. Jauhri et al., “The Llama 3 herd of models,” <https://arxiv.org/pdf/2407.21783>, 2024.
- [4] Z. Cheni, H. Huang, A. Andrusenko et al., “SALM: Speech-augmented language model with in-context learning for speech recognition and translation,” in *Proc. ICASSP*, 2024.
- [5] A. Abouelenin, A. Ashfaq, A. Atkinson et al., “Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs,” <https://arxiv.org/abs/2503.01743>, 2025.
- [6] G. Saon, A. Dekel, A. Brooks et al., “Granite-speech: open-source speech-aware LLMs with strong English ASR capabilities,” in *Proc. ASRU*, 2025.
- [7] Xian Shi, Xiong Wang, Zhifang Guo et al., “Qwen3-asr technical report,” *arXiv preprint arXiv:2601.21337*, 2026.
- [8] E. Ng, C.-C. Chiu, Y. Zhang et al., “Pushing the limits of non-autoregressive speech recognition,” in *Proc. INTERSPEECH*, 2021.
- [9] A. Gulati, J. Qin, C. Chiu et al., “Conformer: Convolution-augmented transformer for speech recognition,” in *Proc. INTERSPEECH*, 2020.
- [10] A. Graves, S. Fernández, F. Gomez et al., “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proc. ICML*, 2006.
- [11] “Open ASR Leaderboard,” https://huggingface.co/spaces/hf-audio/open_asr_leaderboard, Accessed 2 September 2026.
- [12] M. Burchi and V. Vielzeuf, “Efficient conformer: Progressive downsampling and grouped attention for automatic speech recognition,” in *Proc. ASRU*, 2021.
- [13] D. Rekesh, N. R. Koluguri, S. Krizan et al., “Fast conformer with linearly scalable attention for efficient speech recognition,” in *Proc. ASRU*, 2023.
- [14] Y. Zhang, W. Han, J. Qin et al., “Google USM: Scaling automatic speech recognition beyond 100 languages,” <https://arxiv.org/abs/2303.01037>, 2023.
- [15] J. Nozaki and T. Komatsu, “Relaxing the conditional independence assumption of CTC-based ASR by conditioning on intermediate predictions,” in *Proc. INTERSPEECH*, 2021.
- [16] V. Pratap, Q. Xu, A. Sriram et al., “MLS: A large-scale multilingual dataset for speech research,” in *Proc. INTERSPEECH*, 2020.
- [17] X. Li, S. Takamichi, T. Saeki et al., “YODAS: YouTube-oriented dataset for audio and speech,” in *Proc. ASRU*, 2023.
- [18] R. Ardila, M. Branson, K. Davis et al., “Common Voice: A massively-multilingual speech corpus,” in *Proc. LREC*, 2020.
- [19] V. Panayotov, G. Chen, D. Povey et al., “Librispeech: An ASR corpus based on public domain audio books,” in *Proc. ICASSP*, 2015.
- [20] C. Wang, M. Rivière, A. Lee et al., “VoxPopuli: A large-scale multilingual speech corpus for representation learning, semi-supervised learning and interpretation,” in *Proc. ACL-IJCNLP*, 2021.
- [21] I. McCowan, J. Carletta, W. Kraaij et al., “The AMI meeting corpus,” in *Proc. Measuring Behavior*, 2005.
- [22] M. Del Rio, P. Ha, Q. McNamara et al., “Earnings-22: A practical benchmark for accents in the wild,” <https://arxiv.org/pdf/2203.15591>, 2022.
- [23] OpenAI, “gpt-oss-120b and gpt-oss-20b model card,” <https://arxiv.org/abs/2508.10925>, 2025.
- [24] Y. A. Li, C. Han, V. S. Raghavan et al., “StyleTTS 2: Towards human-level text-to-speech through style diffusion and adversarial training with large speech language models,” in *Proc. NeurIPS*, 2023.
- [25] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. ICLR*, 2019.
- [26] K. Jordan, “Muon: an optimizer for hidden layers in neural networks,” <https://kellerjordan.github.io/posts/muon/>.
- [27] N. Srivastava, G. Hinton, A. Krizhevsky et al., “Dropout: A simple way to prevent neural networks from overfitting,” *Journal of Machine Learning Research*, vol. 15, no. 56, pp. 1929–1958, 2014.
- [28] D. S. Park, W. Chan, Y. Zhang et al., “SpecAugment: A simple data augmentation method for automatic speech recognition,” in *Proc. INTERSPEECH*, 2019.
- [29] N. Amsel, D. Persson, C. Musco et al., “The Polar Express: Optimal matrix sign methods and their application to the Muon algorithm,” in *Proc. ICLR*, 2026.
- [30] J. Liu, J. Su, X. Yao et al., “Muon is scalable for LLM training,” <https://arxiv.org/pdf/2502.16982>, 2025.
- [31] J. Bernstein and L. Newhouse, “Old optimizer, new norm: an anthology,” in *OPT2024: 16th Annual Workshop on Optimization for Machine Learning*, 2024.
- [32] D. Snyder, G. Chen, and D. Povey, “MUSAN: A music, speech, and noise corpus,” <https://arxiv.org/abs/1510.08484>, 2015.
- [33] T. Ko, V. Peddinti, D. Povey et al., “A study on data augmentation of reverberant speech for robust speech recognition,” in *Proc. ICASSP*, 2017.
- [34] “Far-Field ASR Leaderboard,” <https://huggingface.co/spaces/treble-technologies/ffasr>, Accessed 2 September 2026.
- [35] P. Wang, “conformer,” GitHub repository, <https://github.com/lucidrains/conformer>.